GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

DEPARTMENT OF INFORMATION TECHNOLOGY



FINAL REPORT

SmartStudy AI: A Hybrid Artificial Intelligence Academic Advisory System

Bachelor of Information Technology (BIT)

Group No: 32

	Member Name	Index Number
1	DR Warnasooriya	D/BIT/24/0062
2	SHRU De Silva	D/BIT/24/0080
3	SHSA Fernando	D/COE/25/0017
4	KPP Pavidura	D/DBA/25/0027

Contents

Abstract……..3

1. Introduction.....Error! Bookmark not defined.

1.1 Background.....Error! Bookmark not defined.

1.2 Problem Statement.....3

1.3 Objectives.....Error! Bookmark not defined.

1.4 Target Users.....4

2. AI Techniques and Dataset Details.....Error! Bookmark not defined.

2.1 Selected AI Techniques.....5

2.2Dataset and Knowledge Representation..... 5,Error! Bookmark not defined.

2.3Streamlit Interface……..6,7

3. Implemented Prototype.....Error! Bookmark not defined.

3.1 System Workflow..... 8

3.2 Prototype Components..... 9

3.3 Dashboard Inputs.....10,11

3.4 Integration Behavior.....11,12

4. Model Development and Results..... 12,13

5. Feature Evidence and Sample Output Interpretation..... 13,14,15

6. Project Status..... 15

7.Limitations and Ethical Considerations……16

8. Conclusion..... 16

9. References.....17

10. Appendix - INDIVIDUAL CONTRIBUTION SHEET.....17

Abstract

The rigorous Bachelor of Information Technology (BIT) curriculum may result in ineffective study habits and academic stress for students. SmartStudy AI is an on-premises academic guidance system created to produce organized study plans that correspond with the 24 fundamental modules of the KDU BIT program. The system employs a hybrid AI architecture that integrates rule-based deterministic checks, organized curriculum knowledge, and a local Llama 3.2 3B language model. A Streamlit interface gathers study limitations, while heuristic guidelines evaluate workload viability prior to generation. The system employs Ollama for local inference and conducts regular-expression sanitization prior to generating a formatted PDF using the fpdf2 library. The project additionally employs programmatically created synthetic student-profile data for regulated development and testing instead of actual student records. Development testing revealed issues with repetitive token generation and PDF rendering; adjustments were made to generation settings and output-cleaning procedures to resolve them. The prototype showcases how AI operating locally can aid in tailored, curriculum-focused academic planning while minimizing the need to send student inquiries to outside services.

1.Introduction

1.1 Background

Artificial intelligence is becoming more commonly utilized to enhance learning, facilitate information retrieval, and aid in academic planning. Nevertheless, general-purpose language models might lack the specific local knowledge necessary to develop study plans that adhere to a particular university curriculum. Students in the KDU BIT program take various modules and need to manage the breadth of each module alongside the time left before exams, their existing skill level, and the number of hours they can feasibly study daily.

SmartStudy AI meets this requirement with a specialized academic advisory system. It integrates organized information regarding the 24 essential KDU BIT modules with a language model that is hosted locally. Students input their study limitations via a user interface, and the system produces a structured plan aimed at assisting them in managing their available study time more efficiently.

1.2 Problem Statement

There is a deficiency of academic planning tools that respect privacy and are tailored to the curriculum, which can create organized and logical study plans for IT undergraduates at KDU. Generic tools might offer recommendations that do not correspond with the structure or content of the KDU modules, while AI services hosted in the cloud can raise concerns about data sharing, latency, and associated costs. Additionally, a key challenge is ensuring that the produced schedules are practical and that the results are formatted neatly for easy use.

1.3 Objectives

1. Create an organized knowledge framework that encompasses the 24 essential KDU BIT modules along with their selected academic resources.
2. Create artificial student-profile data to aid in regulated development and testing.
3. Establish deterministic heuristic safeguards to detect unfeasible or unsafe study-time demands.
4. Implement a locally hosted Llama 3.2 3B language model using Ollama to generate a study roadmap.
5. Utilize prompt design and limited generation parameters to foster organized, pertinent results
6. Develop a Streamlit interface along with a PDF-generation workflow that cleanses model output prior to display.
7. Assess the combined workflow and improve any problems identified during development testing.

1.4 Target Users

Target user	Expected value
KDU BIT undergraduates	Get organized study schedules tailored to your modules, available time, and study capability.
Students preparing for examinations	Arrange revision priorities and daily study tasks within a constrained schedule.
Academic support users	Utilize a curriculum-aware prototype to investigate AI-assisted planning hosted locally.

2. AI Techniques and Dataset Details

2.1 Selected AI Techniques

Technique	Role in SmartStudy AI
Rule-based reasoning / heuristic expert system	Verifies study limitations and workload practicality using guardrails.py prior to model creation.
Knowledge representation	Utilizes organized dictionaries and curriculum details to link 24 KDU BIT modules with selected resources.
Natural Language Processing and deep learning	Utilizes the local Llama 3.2 3B language model to comprehend constraints and produce study-roadmap content.
Parameter-Efficient Fine-Tuning (PEFT) / LoRA	Employed as a strategy for domain adaptation to promote compliance with academic terminology and output formatting.
Constrained text generation	Nucleus sampling, temperature adjustment, and penalties for repetition aid in minimizing redundant or poorly organized results.

2.2 Dataset and Knowledge Representation

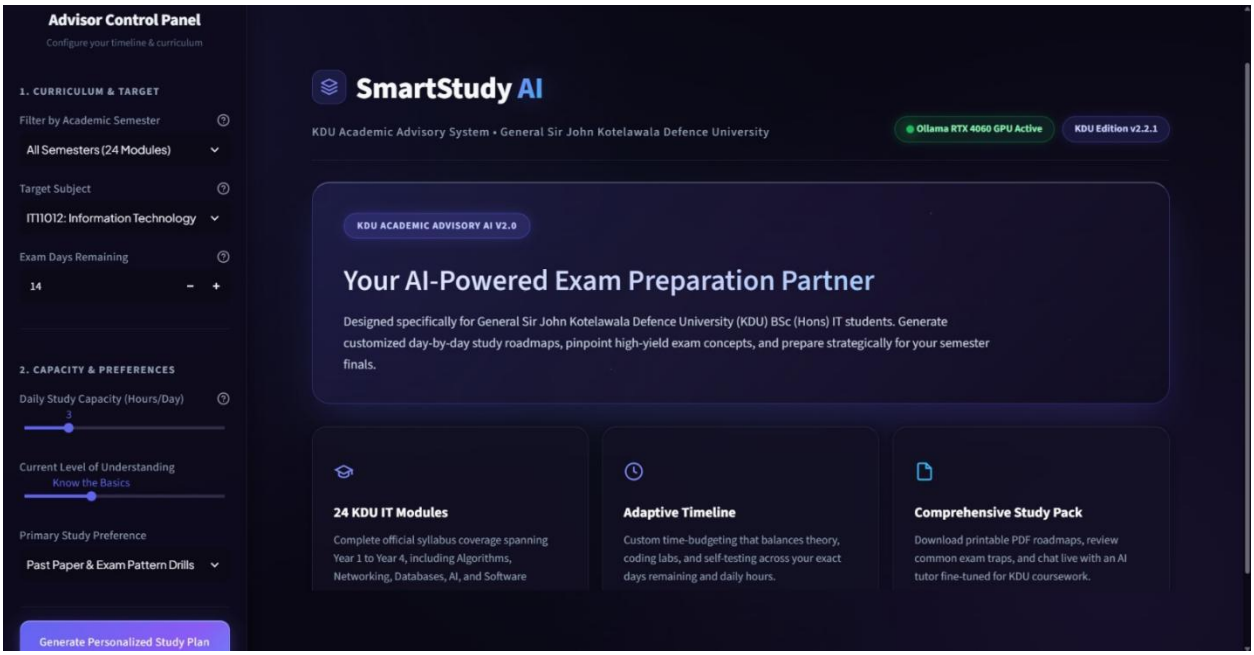
The initiative utilizes automatically created synthetic student-profile data instead of actual records gathered from real students. The synthetic data replicates study-related factors like study hours, attendance rate, and prior GPA, enabling the team to test various input scenarios in a regulated setting.

A distinct collection of organized domain dictionaries embodies the scholarly knowledge utilized by the application. These dictionaries encompass 24 essential KDU BIT curriculum modules and link the modules to selected resources, such as KDU Library and Coursera materials. Distinguishing simulated student characteristics from curriculum knowledge clarifies the data structure and simplifies the maintenance of domain information.

Data component	Description	Purpose
Synthetic student profiles	Automatically created study-related metrics including Study_Hours, Attendance_%, and Previous_GPA.	Evaluate application functionality and input boundary scenarios without utilizing actual student data.
Curriculum dictionaries	Organized data for 24 fundamental KDU BIT subjects.	Grounded plans generated within the desired university curriculum.
Academic resource mappings	Compiled materials related to module understanding.	Assist the system in directing students to pertinent educational resources.

2.3 Streamlit Interface

The Streamlit interface is created to offer an easy-to-use and straightforward platform for the SmartStudy AI system. It enables KDU BIT students to input their academic information, choose applicable curriculum modules, and share details about their study needs. Using the given inputs, the system creates customized study plans, educational suggestions, and syllabus-related clarifications by employing the built-in rule-based advisory feature and the locally operating Llama 3.2 language model. The interface displays the produced results in an organized manner, assisting students in managing their study tasks and grasping intricate academic ideas more efficiently. Furthermore, the interface enhances the entire workflow by linking user inputs to the system's academic knowledge base, safety measures, and response-generation elements. The Streamlit interface acts as the primary way for students to interact with the SmartStudy AI system, offering convenient academic support without needing a constant internet connection for local AI processing.



3. Implemented Prototype

The prototype integrates a Streamlit user interface, a validation component, a locally hosted language model, output sanitization, and PDF creation. This process illustrates the route from student-submitted structured study plan. The system is intended to function locally with Ollama and Llama 3.2 3B.

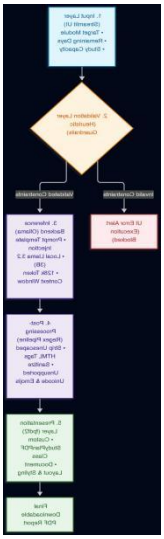


Figure 1: Prototype of the SmartStudy AI Academic Advisory System

3.1 System Workflow

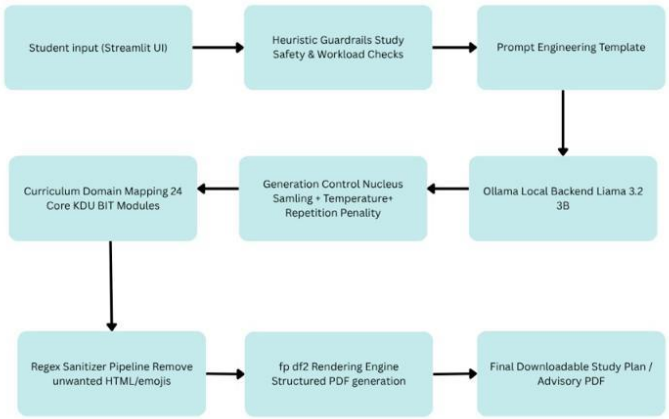


Figure 2: System Workflow of the SmartStudy AI Academic Advisory System

3.2 Prototype Components

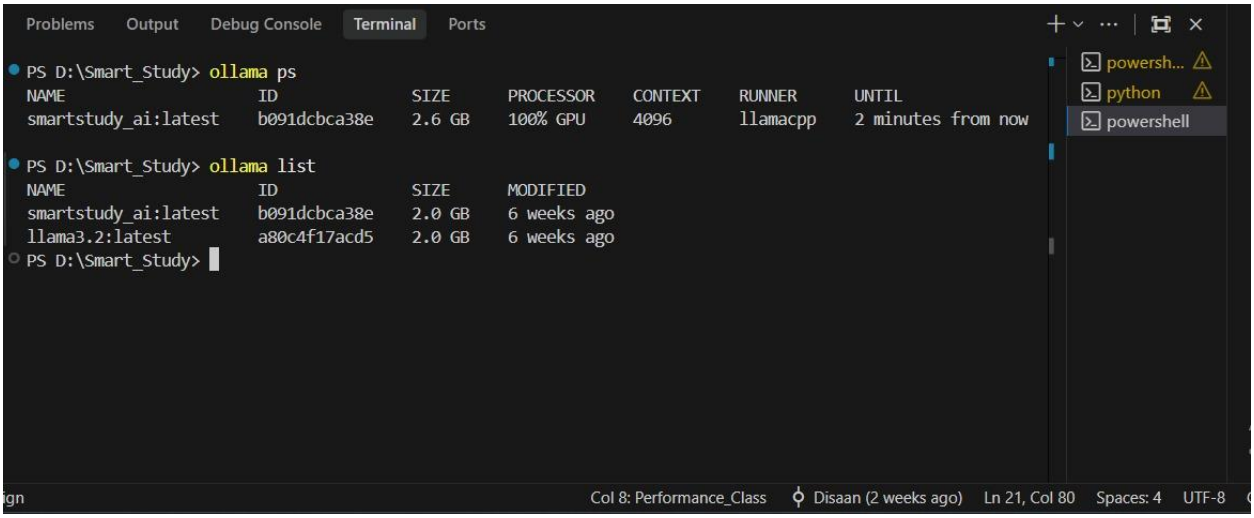


Figure 3: Local Llama 3.2 Model Running Through Ollama

1. Streamlit interface: offers the user-oriented form for defining module and study limitations.
2. Heuristic boundaries: confirm the practicality of the proposed study timetable prior to creation.
3. Local inference backend: utilizes Ollama to operate Llama 3.2 3B without transmitting the inference request to a cloud AI API.
4. Organized curriculum knowledge: provides module-specific context and organized academic-resource connections.
5. Output sanitization: employs Regex rules to sanitize unprocessed generated text prior to its access by the PDF renderer.
6. PDF creator: employs fpdf2 and the StudyPlanPDF styling class to generate the final study-plan document.
7. Synthetic data generator: produces artificial student-profile data for regulated development assessments.

3.3 Dashboard Inputs

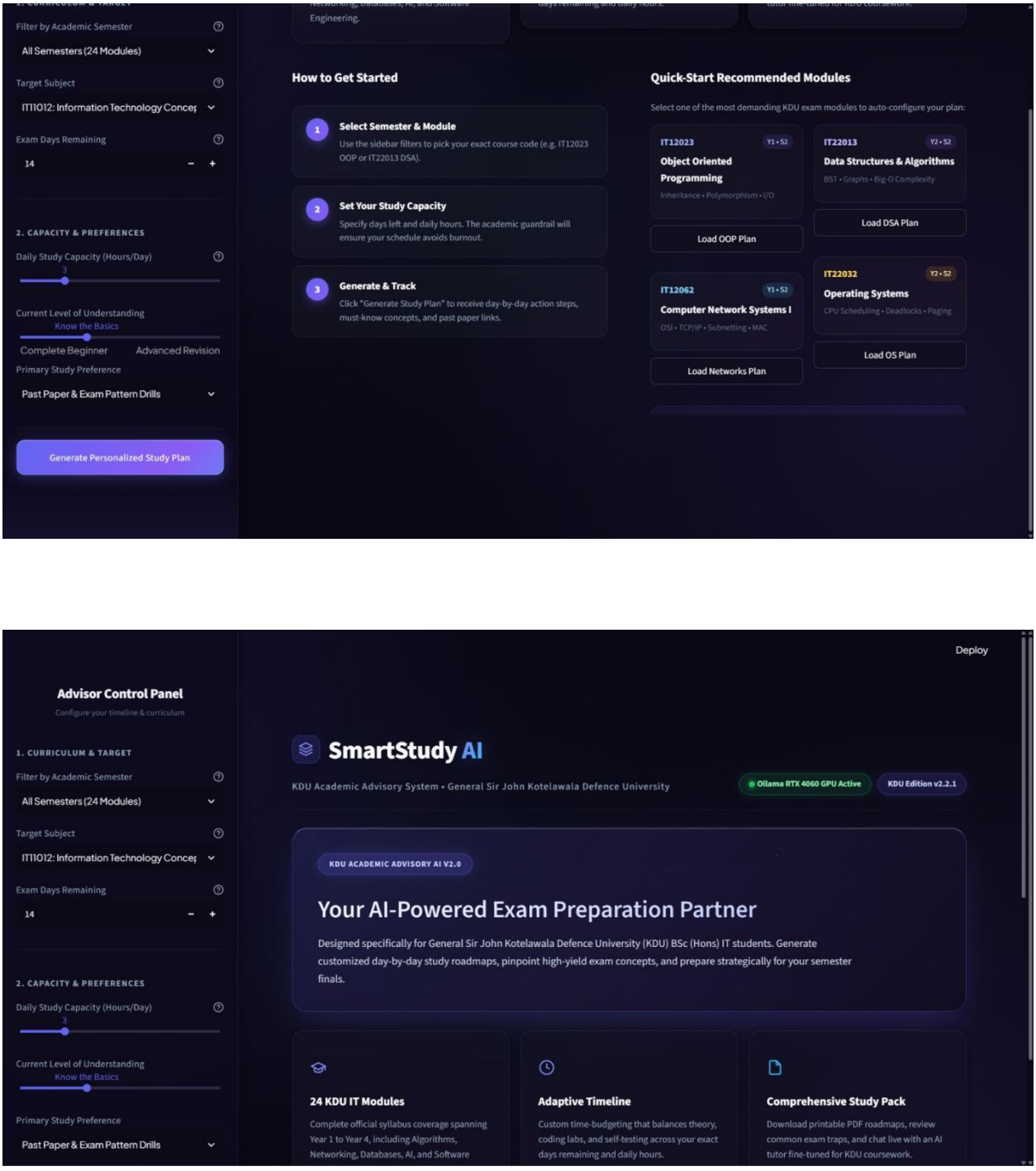


Figure 4: Dashboard Showing Student Input Parameters for Study Plan Generatio

Input / output	Description
Target module	The KDU BIT module for which the student wants a study roadmap.
Remaining days	The time available before the target study deadline or examination.
Daily study capacity	The amount of study time the student can realistically allocate per day.
Validation response	An error or prompt when supplied constraints are invalid or infeasible.
Generated roadmap	Structured study guidance based on the validated constraints and curriculum context.
PDF document	A formatted, multi-page output generated from sanitized model text.

3.4 Integration Behavior

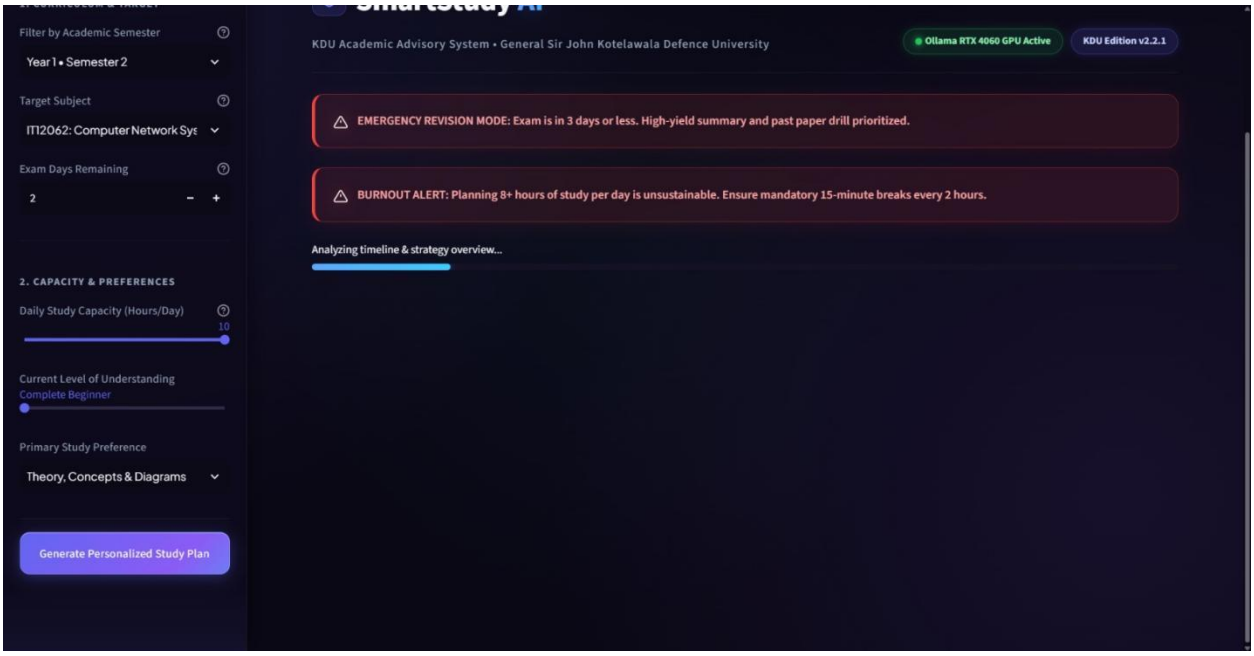


Figure 5: Input Validation and Heuristic Guardrails

The combined process verifies user requirements prior to calling the language model. This design holds significance as a generative model might not consistently uphold rigorous time arithmetic on its own. If a request breaches the established guardrails, the interface can display a validation message rather than forwarding an unfeasible schedule to the generation phase.

Following inference, the output is not sent straight to the PDF renderer. A cleaning phase based on Regex eliminates troublesome markup and unsupported characters that might otherwise impact PDF generation. The sanitized material is subsequently generated with fpdf2 and the tailored StudyPlanPDF class. This division of validation, generation, cleaning, and presentation simplifies testing and maintenance of the workflow.

4. Model Development and Results

The main application logic was developed in Python. Llama 3.2 3B was chosen as the local language model and utilized via Ollama. The project outlines the implementation of parameter-efficient adaptation through LoRA to improve the model's alignment with the specialized terminology and structure anticipated by the academic advisory system.

Generation controls were set up to promote consistent and organized results. The project documents specify a top_p value of 0.9, temperature configurations in the 0.4–0.7 interval, and repetition penalties between 1.1 and 1.2. These controls were implemented with prompt engineering to minimize repetitive outputs and enhance conformity to the desired format.

Issue observed during testing	Adjustment described in the project
Repeated token patterns, such as Greek-alphabet cycles when num_predict was configured too high.	Adjusted repetition penalty and refined generation/context parameters.
Raw HTML elements and emojis disrupted PDF rendering.	Implemented Regex-based cleansing to eliminate problematic markup and unsupported characters.
Base PDF fonts did not consistently provide support for special academic symbols.	Implemented personalized character management / byte-mapping for unique symbols.

The project report details that the output-cleaning modifications address the PDF rendering issues faced during the development phase. The provided project details do not include any numeric accuracy score or formal benchmark metric; therefore, this report does not give a numerical performance result to the language model.

5. Feature Evidence and Sample Output Interpretation

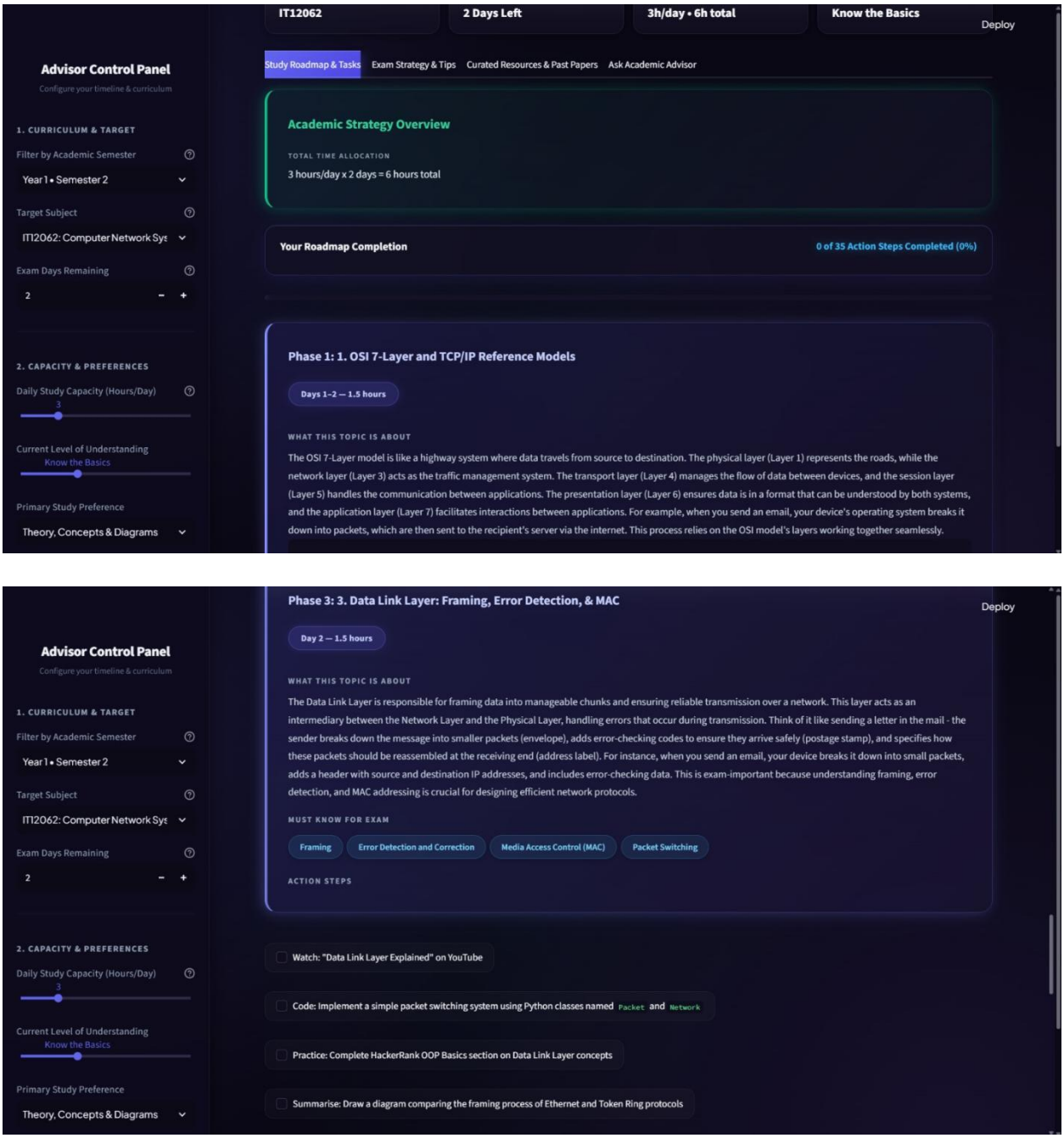


Figure 6: AI-Generated Study Roadmap

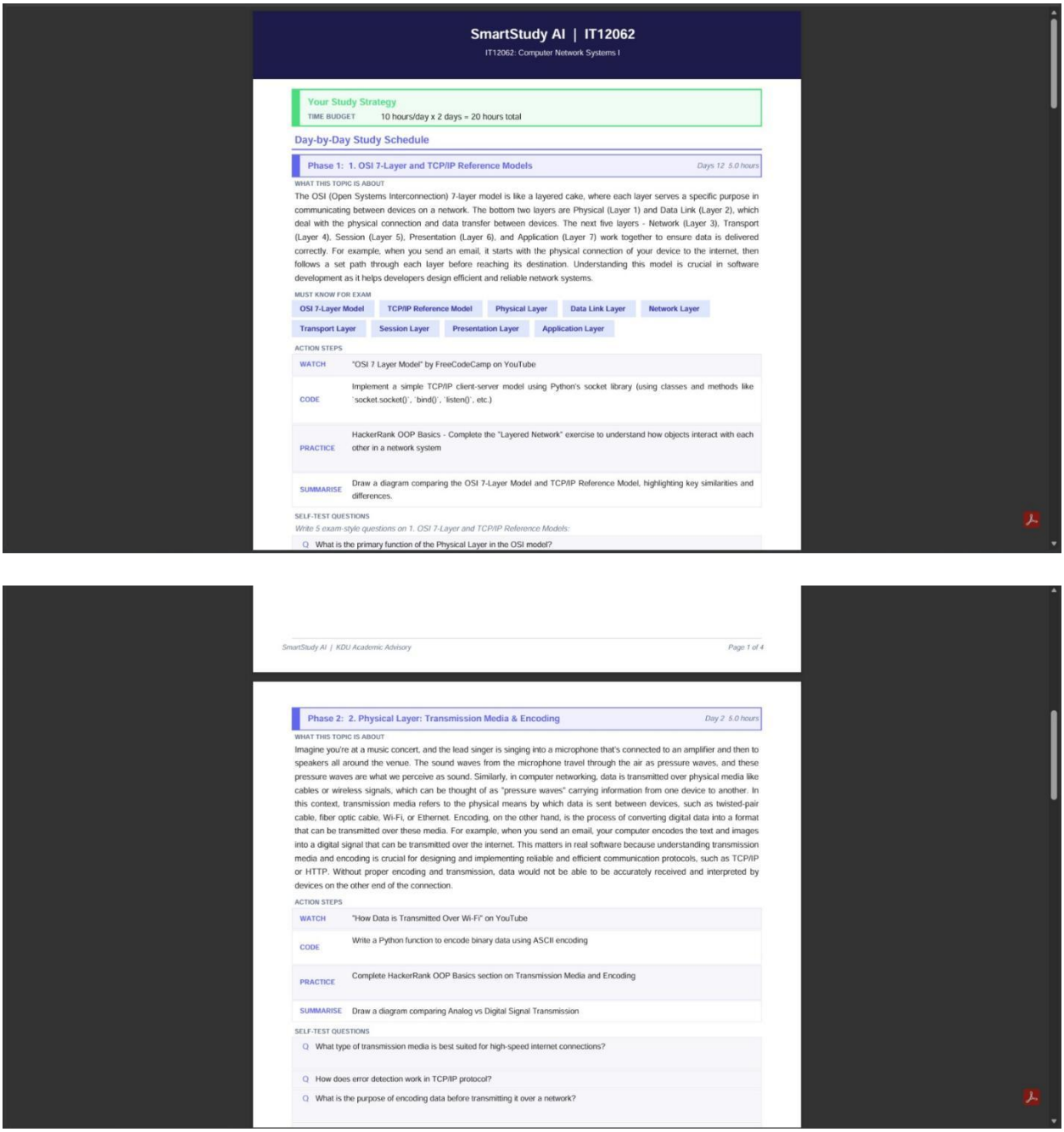


Figure 7: Final PDF Study Plan Generated by the System

The main functional proof for SmartStudy AI lies in the complete workflow: the user provides a module and realistic study parameters, the guardrails assess the request, the local model creates the plan, and the PDF pipeline transforms the refined response into an organized document.

Test scenario	Expected behaviour
Feasible study request	Receive the input and create an organized study plan according to the given limitations.
Infeasible workload request	Decline or mark the request via the heuristic validation layer prior to generation.
Model response contains HTML or unsupported characters	Clean the reply prior to dispatching it to the PDF creator.
Generated plan is ready for export	Generate a formatted PDF with the StudyPlanPDF class.

6.Project Status

Project component	Status based on supplied project details
Structured knowledge for 24 KDU BIT modules	Prepared / established
Synthetic student-profile dataset	Generated programmatically
Heuristic validation middleware	Executed in guardrails.py
Local Llama 3.2 3B inference using Ollama	Integrated in the workflow
Prompt and generation-setting refinement	Performed during testing
Regex-based output sanitization	Implemented
PDF generation using fpdf2	Implemented
End-to-end prototype workflow	Described as integrated
Numerical model benchmark results	Not provided in supplied details
Screenshots of the final running interface	Not included in supplied details

7. Limitations and Ethical Considerations

7.1 Limitations

- The 3B parameter language model may be less capable than larger models on highly complex, multi-step reasoning tasks.
- Synthetic student-profile data supports controlled testing but does not represent the full range of real student behaviour.
- Local inference speed depends on the hardware available on the computer running the application.
- Generated study advice should be treated as planning support and checked by the student for relevance and feasibility.
- The supplied project details do not include numeric benchmark results, so quantitative claims about model accuracy cannot be made.

7.2 Ethical and Social Considerations

A primary objective in the design is local processing. Executing inference on the host machine via Ollama minimizes the necessity of sending student prompts to an outside AI service. Employing synthetic profiles also eliminates dependence on actual student records throughout the outlined development process. The heuristic guardrails aim to deter impractical study schedules and emphasize realistic workload limits. The system needs to be portrayed as a scholarly assistance resource instead of a substitute for student decision-making or official university advice.

8. Conclusion

SmartStudy AI showcases a blended method for curriculum-aware academic planning by integrating fixed rules, organized KDU BIT knowledge, and a locally operated language model. The Streamlit platform records student limitations, the heuristic component evaluates workload viability, and Ollama executes the Llama 3.2 3B model to create study roadmaps. Post-processing with regex and fpdf2 offers a way to transform raw generated text into a formatted PDF file.

The development process also revealed practical difficulties in managing language-model results and ensuring that output is consistent. Prompt and decoding modifications were implemented to minimize repetitive output, while tailored sanitization techniques tackled markup and character problems in PDF generation. The existing prototype establishes a basis for additional assessment. Future efforts need to encompass wider test scenarios, established evaluation standards, further curriculum assessment, and images or captured outputs from the fully operational system.

9. References

[1] Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/

[2] Ollama, “Ollama Documentation.” Available: https://docs.ollama.com/

[3] Meta, “Llama 3.2 model.” Available: https://www.llama.com/

[4] PyFPDF / fpdf2, “fpdf2 Documentation.” Available: https://py-pdf.github.io/fpdf2/

[5] Hugging Face, “PEFT: Parameter-Efficient Fine-Tuning.” Available: https://huggingface.co/docs/peft/

[6] General Sir John Kotelawala Defence University, Bachelor of Information Technology programme curriculum and module materials.

Appendix - INDIVIDUAL CONTRIBUTION SHEET

APPENDIX A — INDIVIDUAL CONTRIBUTION SHEET

Member	Registration number	Contribution described in supplied details
DR Warnasooriya	D/BIT/24/0062	Architected the core AI workflow, implemented heuristic guardrails, integrated the Llama 3.2 backend, and engineered the PDF export pipeline.
UHR Umasha	D/BIT/24/0080	Managed documentation of project reports and contributed to structuring the curriculum knowledge datasets.
SHSA Fernando	D/COE/25/0017	Led synthetic dataset generation and carried out fine-tuning and training work for the core machine-learning model.
KPP Pavidura	D/DBA/25/0027	Designed and developed the interactive frontend user interface using Streamlit.